

Exercise 2: Scripts and Commenting, Formatted Output and Online Help

Note: Before you start this exercise, if you already have Octave running and some variables defined, remove them with the “clear” command. You should also make sure that the File Browser is currently set to the directory you want to work in and save files to. One way to stay organized is to create a general Octave working directory, and then make a new folder within your Octave directory that stores each day’s work. In the Griffin group we often organize files and folders by labelling them with a numerical date in the format YYMMDD. For example, a folder containing work from January 23, 2017 would be named “170123”. This makes it easy to find our files when looking back at past work, especially if we are keeping a good research notebook with dated entries describing each day’s work.

1. The Script Editor

So far we have executed a few commands one at a time through the Command Window. This works for very simple programs, but if we eventually want to do some science we will need to send the computer lots of commands at once. To do this we will use a tool called the Script Editor. At the bottom of the Command Window you will see several tabs: Command Window, Editor, and Documentation. Click on the Editor tab. This takes us to a text editor that we can use to create text files holding a sequence of commands for the computer. We call these files scripts. Working with scripts allows us to run lots of commands at once, and also helps us read through our script to figure out how the program is working. Across the top of the Script Editor there are bunch of buttons for working on our script file. Find the buttons to open and save script files, create a new script, and a button that will save your current file and then run your script. If you let your cursor hang over the buttons, explanatory text will pop up.

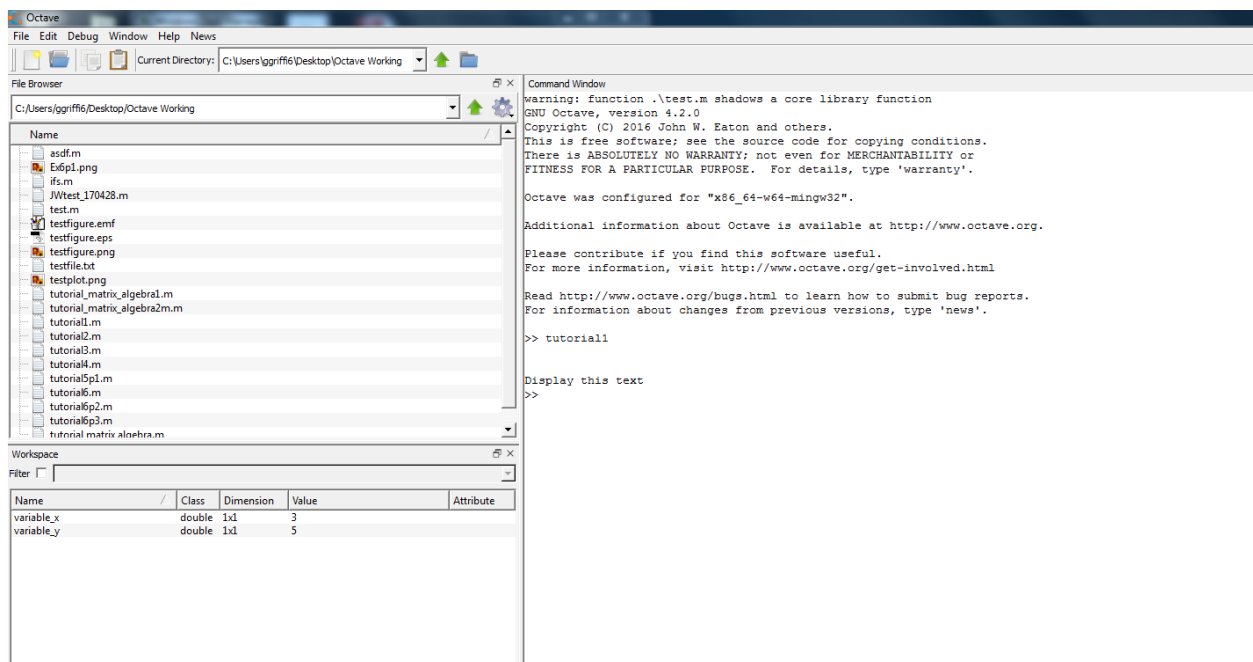
We are ready to write a script and run it. Copy and paste the following text into the Script Editor.

```
%file created by XXX on YYMMDD
%tutorial first script
clear;
printf('\nDisplay this text\n');
variable_x = 3;
variable_y = 5;
```

Griffin Lab Programming Tutorial

Put your initials into the space designated XXX and the current date into the YYMMDD field. Otherwise copy the text exactly as it appears. Save your script under the name “tutorial_1.m”. Octave should automatically enforce the .m file suffix, Octave scripts all have this designation.

You should see the lines that start with % symbols turn green, and the text ‘Display this text’ turn orange. If the colors don’t change properly, you may need to retype the % or ‘ symbols in the Script Editor. When everything looks right, click the save/run button (gear with a yellow triangle) to run your script. Keep your tutorial.m file name. Now click over to the Command Window. It should look like this:



When you told Octave to run your script, it basically typed the script name into the command window and hit enter. That is why you see the text “tutorial1” at the first prompt, followed by an empty space. Then the program ran. To run your program, the computer simply executed the sequence of commands you provided, starting from the top and going in order downwards.

The first two lines of your script, colored green and starting with % symbols, are called comments. Comments are text that you add to your program to explain how it works or highlight key information for yourself or other programmers. In Octave we identify text as a comment using a % symbol, and the computer will know that anything following the % symbol is not a

command that it should execute. The GUI program turns all comments green so you can see them easily.

The first line containing a command was line 3, the clear command. When the computer executed this command any variables stored in the workspace were cleared from memory. It is good practice to start your programs this way. This practice eliminates avoid hard-to-detect problems due to previously stored variables, and keeps the workspace from getting cluttered while you run multiple scripts.

The next command the computer encountered was the printf command on line 4. This is an Octave command for displaying text output to the Command Window. If you look in the command window, you can see the text output. The printf command is a very versatile tool that allows you to display all kinds of formatted output. Here we use it in the simplest way possible, to spit out a hardcoded (i.e. specified in the script directly) string of text. We use the `\n` character to instruct the printf character to move to a new line. We have to use a special code for this because we can't just hit enter within our string of text, the computer won't know what to do with that information. Instead, we put `\n` into our text anywhere we want to move to a new line. This method is called using an escape sequence. There are escape sequences for displaying many other types of characters that can mess up a line of computer code, such as slashes and quotation marks. We put a newline escape sequence at both the beginning and the end of our text string, so that the text would be displayed by itself on one line.

Next, the computer sees the two lines for assigning values to `variable_x` and `variable_y`, and executes them. If you look in your workspace you can confirm that this part of your script worked.

2. Formatted Output, Online Help

Let's say we want to add a line to our program that displays the result of multiplying `variable_x` and `variable_y`. I can multiply the two variables by adding the following line to the end of our program:

```
result = variable_x*variable_y
```

Before we run the program, save it under a new name (“tutorial2.m”). The best practice for managing files when writing computer code is to never save over an older version, in case we want to go back to it. For this reason we should always save under a new file name. Our computer programming scripts use up tiny amounts of memory, there is no reason to ever delete or save over them. Hard drive space is abundant and therefore expendable, never save over a working program! It is always nice to have previous working versions to fall back on if you break your program while altering it.

The new line tells above the computer to create a variable called “result”, and then assign to that variable the value of `variable_x` times `variable_y`. If you have left the semicolon off the new line, as shown above, the computer will report the value of the result variable in the command window when you run your program. This is one way to get output from the program. If we want to report the value of several numbers at once, or of some mathematical relationship between several variables, we might want clearer formatting for our output. We can achieve this with `printf`.

Let’s dig into the `printf` command a little deeper, and use that as a way to learn how to go online looking for help. Google “Octave printf”. One of your top results will be “GNU Octave: Formatted Output”, with a web address starting with “www.gnu.org”. This is the online manual for Octave, and it is a great source for learning about how commands work. If you really want to become an expert programmer, one way to do that would be to click on the Contents link on this page, and work your way through the whole manual. Our research tutorial is a bit more of a guided look at some of the tools we use most frequently. Many other web pages are useful and good, but if you are trying to figure out a new command the official gnu.org page is your best bet. Read through this page to get an idea of how the `printf` command works.

We already know how to get `printf` to output a string of text. The `printf` command will try to display everything inside the single quotes. To insert number into the text string, we insert a placeholder symbol, such as `%f` or `%d`. The various types of placeholders for formatted output are described on the gnu.org page you just looked through, read carefully! The placeholder tells `printf` to put a number into the text string with formatting specified by the letter (f for floating point, d for integer, e for scientific notation, etc.). After the text string, but still inside the parentheses of the `printf` command, we place a comma to separate our text string from the next

input we want to give printf. Next we provide the number we want inserted into the text string. This looks like:

```
printf('The value of x times y is %f', result);
```

This formatting, a command name followed by parentheses containing multiple items separated by commas, is very common in computer programming languages. We call the items inside the parentheses “arguments”, and we say that we are “passing them to” the printf function. Our command tells Octave to output the text string, containing the value of the result variable in floating point form. To test this out, modify your tutorial2.m script so that it looks like the code below. Run it to see the result.

```
%file created by XXX on YYMMDD  
clear;  
variable_x = 3;  
variable_y = 5;  
result = variable_x*variable_y;  
printf('The value of x times y is %f\n', result);
```

If you click the gnu.org Octave website to through the next few sections (through section 14.2.10), you can learn more about formatting numerical output using printf. Read carefully, this information will be useful for the practice exercises below.

3. Practice

Play with tutorial2.m it a bit.

- a. What happens when you change the letter in the placeholder to d or e? What happens if I change the placeholder from %f to %2.3f or %10.3f. What happens when I use %.1e or %.3e?
- b. What happens if I remove the result variable from the printf line. What if I replace it with “variable_x*variable_y”? Can you run the program and display this value without creating the variable result?
- c. Google online help for the command strcat, which performs string concatenation. Basically this means adding two pieces of text to make one larger piece of text. Also look up the function num2str, which takes a numerical value and converts it into a text string. This is actually a fairly complex operation for a computer to

perform (more about bits and how the computer handles them). Can you use these two commands to create a version of your program that displays the same output as the one above using a single text string (i.e. without using a placeholder to insert the number)?

2. The known electronic energy levels of all materials (official US government reference data) can be found at: http://physics.nist.gov/PhysRefData/ASD/levels_form.html To get to the energy level table for Hydrogen, type H into the Spectrum box and click Retrieve Data. The energy levels are listed in a table. The familiar general chemistry designations are on the left (1s, 2s, 2p, etc ...). The energy value corresponding to each energy level is listed in a unit called wavenumbers (cm^{-1}). There are a few other columns in the table that you can ignore, they give other properties of H electrons in these orbitals that you can learn about in CHE 302.
 - a. Using what you know about formatted output, write a script that can display a table of the energies for the energy levels of H atoms up through $n = 6$ in the command window. For $n > 1$, use the designation that does not specify the angular momentum quantum number l . (e.g. for $n = 2$, read the row on the table with the entry “2” in the first column, not the “2s” or “2p” rows.) How good can you make your table look in this simple interface?
 - b. Look up conversion factors to convert cm^{-1} into the energy units hartrees, electronvolts, and joules. Modify your program so that it displays the Hydrogen energy levels in all of these units in one table, with each energy unit as a column.